RESEARCH PLAN · RANSKNOW DATASET

From keyword extraction to a benchmarked, uncertainty-aware dataset

An experimental plan for a paper on RansKnow: how it was built, what it actually contains, and what a rigorous set of baseline, calibration, conformal, and out-of-distribution experiments looks like on top of it.

1,128	1,034	92	21	92.7%
video records	with transcripts	channels	families detected	ASR-transcribed

CONTENTS

FRAMING

Why this plan, not the usual one

The obvious next step after the Knowledge Agent is "train some classifiers on the features and report accuracy." That paper gets rejected, or worse, gets accepted and is wrong, because the labels the classifiers would be trained on *and* evaluated against are the same regex-matched labels the Knowledge Agent produced — there is no

independent ground truth in the loop anywhere. Reporting 90% F1 against your own keyword matcher's opinion of itself is not a result.

So this plan is ordered deliberately: **Phase 0 exists before any modeling**, because two audits of the current labels turned up real, checkable problems that would otherwise quietly become the paper's biggest weakness if a reviewer found them instead of you. Everything after that — the model ladder, calibration, conformal prediction, OOD detection — is standard and well-trodden. The label-quality audit and the small human-annotated evaluation set are the part that makes the rest of it trustworthy, and arguably the part most worth writing about.

SECTION 1

Two findings that reshape the plan

FINDING A — GENERIC-WORD ALIAS COLLISIONS

`Play` is the single most-detected ransomware family in the dataset — 386 of the 494 family-tagged videos (78%) carry it, dwarfing every other family. It is also an English word that appears in ordinary sentences ("let's *play* the video," "just *play* with it") far more often than in ransomware discussions. I manually checked 8 `Play`-tagged videos at random: **zero** were actually about Play ransomware. The same pattern shows up for `Hive` (collides with "registry hive," a routine DFIR term — 2 of 4 spot-checked hits were the registry, not the group) and `Cactus` (collides with CactusCon, the conference — 1 of 4 hits was the conference name). The alias table (`rubrics/Ransomware_Family_Coverage_List.xlsx`) adds the bare family name as a matchable alias for every family, with no requirement for disambiguating context, so any family whose canonical name is also a common word or overlaps a security-adjacent term inherits this failure mode for free.

FINDING B — MOST "TRANSCRIPTS" ARE WHISPER OUTPUT, NOT CAPTIONS

Only 74 of 1,034 transcripts (7.2%) came from official YouTube captions via the API. **959 (92.7%) are ASR transcriptions** — 554 from `whisper_base` (the smallest, least accurate Whisper checkpoint) and 405 from a local Whisper fallback. Aggregate detection rates don't show an obvious gap between providers (family-detection rate is

actually similar across all three — 44–49%), so this isn't clearly suppressing recall in the aggregate. But `whisper_base` is known to mis-transcribe rare proper nouns and jargon — exactly the vocabulary the Knowledge Agent's regexes are built to catch (`LockBit`, `Mimikatz`, `rclone`). This needs a targeted spot-check rather than an aggregate-rate argument, because the errors that matter here are the kind that wouldn't move an aggregate rate much while still being individually consequential.

FINDING C — `TRANSCRIPT_PATH` IS NOT PORTABLE

The feature CSV stores absolute filesystem paths from whatever machine happened to run `knowledge_agent.py` at the time (e.g. `/tmp/claude-1004/.../RansKnow_repo2/transcripts/C01.../V0001.txt` next to plain `/home/henrykabuye/RansKnow/transcripts/...` rows in the same column). This is a five-minute fix — regenerate the column as a path relative to the repo root — but it should happen before anyone else tries to reproduce a result from the CSV alone.

SECTION 2

Dataset snapshot

Numbers below are computed directly from `outputs/Knowledge_Agent_Features_1034.csv`, not estimated.

Coverage by year

YEAR	VIDEOS	YEAR	VIDEOS
2016	3	2022	46
2017	1	2023	88
2018	1	2024	180
2019	12	2025	299
2020	29	2026	338
2021	37	**Total**	**1,034**

79% of the dataset (817/1,034) is from 2024 onward — this is a recency-skewed corpus, which is exactly what makes a train-on-old / test-on-new temporal split meaningful rather than an afterthought (see Phase 4).

Family, tactic, platform, tool base rates

SIGNAL	DISTRIBUTION
Family_Count	0 families: 540 (52.2%) · 1: 364 · 2: 84 · 3: 30 · 4: 10 · 5: 2 · 6: 4
Top families	Play 386 · Qilin 93 · Conti 33 · Royal 28 · LockBit 27 · Akira 24 · Hive 23 · Cl0p 11 · Maze 11 · INC 10 (21 unique total)
Dominant_Tactic	Initial_Access 474 · Impact 179 · *none* 122 · Execution 97 · C2 52 · Discovery 32 · others < 30 each
Platform_Signal	*none* 487 · Windows 416 · Linux 81 · ESXi 50
Tool_Total_Mentions	Any tool mentioned: 105/1,034 (10.2%) · median = 0 · max = 54
DurationSeconds	mean 1,914s (~32 min) · median 1,770s · range 300s–17,618s
Transcript_Provider	whisper_base 554 · local_asr_whisper 405 · youtube_api 74 · ytdlp_captions 1

The shape to plan around: family and tool labels are sparse and long-tailed (single-digit counts for most of the 21 families), tactic/platform labels are dominated by two or three majority classes with a large "none detected" bucket, and everything sits on top of a corpus that is unevenly split across 92 channels (up to 20 videos each) and skewed hard toward 2024–2026.

SECTION 3

Task taxonomy

Five predictive tasks fall directly out of the existing schema. All five currently use Knowledge Agent output as their *only* label source — Phase 0 is what earns the right to call any of them ground truth.

TASK	TYPE	CLASSES	LABEL SOURCE TODAY
Ransomware family	multi-label	21	alias regex match

TASK	TYPE	CLASSES	LABEL SOURCE TODAY
Dominant MITRE tactic	multi-class (+none)	11	max of 10 tactic-pattern counts
Platform signal	multi-class (+none)	4	max of 3 platform-pattern counts
Tool mentions	multi-label	8	per-tool regex match
Ransomware-relevant screen	binary	2	Family_Count > 0 *or* Tactic_Impact > 0 (proposed)

The fifth task isn't in the current schema — it's worth adding because a fast, cheap "is this video even about ransomware" screen is the actual bottleneck if you ever want to point the pipeline at a new batch of channels without hand-reviewing every keyword hit, and it's the easiest task to get a clean gold label for.

`Phase 0 — before any modeling`

Data & label-quality audit

0.1 — Fix the mechanical issues

- Regenerate `Transcript_Path` as repo-relative in the feature CSV (Finding C).
- Add a `Channel_Type` column (Vendor / Conference / DFIR-Independent / Government / Consulting) derived from the channel registries — needed for the GroupKFold-by-type split in Phase 1 and the channel-shift OOD setup in Phase 4.

0.2 — Systematically re-audit the alias list

Don't just fix `Play`. Run every alias in `Ransomware_Family_Coverage_List.xlsx` against a common-English-word check (dictionary lookup, or unigram frequency in a general corpus) and flag any alias that is itself a dictionary word with no ransomware-specific qualifier. For each flagged family, require a co-occurring disambiguator within a short window (e.g. `ransomware`, `group`, `gang`, `encrypt` within ±15 tokens) before counting a hit. Re-run extraction and report the before/after family-count table

— the delta on `Play`, `Hive`, and `Cactus` specifically is very likely a headline number in the paper.

> **EXPERIMENT 0.2**
>
> | | |
> |---|---|
> | Method | Dictionary-collision scan over 41 aliases + windowed-context re-extraction |
> | Output | Precision estimate per flagged family, before/after count table |
> | Cost | ~1 day, no annotation needed |

0.3 — Build a gold evaluation subset

Stratify a sample of ~180 videos across three axes — channel type (Vendor / Conference / Independent / Gov), year bucket (≤2022 / 2023–24 / 2025–26), and current `Family_Count` (0 / 1 / 2+) — then label each video for: does it discuss a ransomware incident (Y/N), which family if any (open text, then reconciled to the alias list), dominant tactic, and platform. **This subset is held out from all training in every later phase** — it exists only to answer "how good are these labels, and how good are the models against real labels, not against the labels the models were trained to reproduce."

Revised from 2 annotators to 1: the original design called for two independent annotators with adjudication and Cohen's κ per field. Recruiting external annotators turned out to require institutional ethics approval this project doesn't have, so the set is annotated by a single investigator instead, working blind to the Knowledge Agent's own label per field until after recording a judgment. This is a disclosed limitation, not a hidden one — single-annotator labels lack inter-annotator-agreement evidence and may carry undetected systematic bias, but they're still independent of the weak-label extraction pipeline in the sense that actually matters here: the label didn't come from the same regex process being evaluated.

> **EXPERIMENT 0.3**
>
> | | |
> |---|---|
> | Sample | ~180 videos, stratified 4×3×3 |
> | Annotator | 1 (single investigator — see note above) |
> | Report | Confusion vs. Knowledge Agent labels; single-annotator limitation discussed explicitly |
> | Use | Held-out eval only, never training — reused in Phases 1–4 |

0.4 – ASR jargon spot-check

From the gold subset, pull the whisper-transcribed videos and manually check a fixed term list (`LockBit` , `Mimikatz` , `PsExec` , `Cobalt Strike` , `rclone` , `BloodHound` , family names) against the source audio for transcription accuracy. This turns Finding B from a plausible concern into a measured word-error-rate-on-jargon number.

Phase 1

Baseline model ladder

Three feature representations, five model families, evaluated identically across all five tasks. The rule-based Knowledge Agent itself is baseline zero — it's a strong lexical baseline, not a strawman, and should be reported as one.

REPRESENTATION	FEEDS
Structured	tactic/tool/platform counts, duration, year, channel type — the existing Knowledge Agent columns, minus the label being predicted
TF-IDF	bag-of-words over full transcript text, 1–2 grams
Embedding	pooled sentence embeddings over transcript chunks (any small open embedding model run locally)

MODEL	INPUT	ROLE
Knowledge Agent (rule-based)	raw text	baseline 0
Logistic Regression	structured or TF-IDF	linear baseline
Linear SVM	structured or TF-IDF	the standard strong baseline for TF-IDF text classification specifically
Random Forest	structured or embedding	also the easiest source of a free uncertainty signal (Phase 2)
Gradient-boosted trees	structured or embedding	usual tabular ceiling

MODEL	INPUT	ROLE
Fine-tuned compact transformer	raw text	text-native ceiling

FRAMEWORK BUILT AND SMOKE-TESTED

The task/feature/model/split registry described above is implemented (`Scripts/rk_pipeline/`) and has actually been run against the real dataset — not just designed. Structured-feature results, 30 task×model×split combinations: the rule-based baseline scores a tautological 1.000 on both tasks (expected — it's being checked against its own labels, see the metrics note below), and the ceiling model per task differs in a way that's itself informative: GBM reaches 0.81–0.83 macro-F1 on `Dominant_Tactic` from structured features alone, while every model tops out under 0.08 macro-F1 on `Family` from the same features. Tactic identity is close to a direct function of the tactic-count columns; family identity isn't — it lives in which words appear, which is the argument for why the TF-IDF/embedding/transformer representations matter specifically for the family task, not just as a formality.

Metric note: macro-F1 over present classes, not the full vocabulary

With 21 long-tailed family classes and 5-fold CV, most folds don't contain every class. Scoring an absent class as F1=0 — sklearn's default — measures how unlucky the fold split was, not model quality. Macro-F1 here is computed only over classes with at least one true instance in that fold's test set (`macro_f1_all_classes` is kept alongside it so the gap stays visible). This is what makes the rule-based baseline's self-consistency check actually read as 1.000 instead of an uninterpretable ~0.7–0.86.

Class balancing

Three rungs, tried in this order — not because deep generative augmentation is unavailable, but because the data doesn't support it yet for the classes that need help most:

METHOD	STATUS	NOTE
`class_weight="balanced"`	implemented, always on	free — no synthetic data, works for LogReg/SVM/RF; LightGBM uses `is_unbalance=True`
SMOTE / ADASYN	implemented	structured features only, per-class adaptive `k_neighbors` (default k=5 would simply error below 6 real examples — several classes sit at exactly 5–6)
CTGAN	registered, not installed	ablation against SMOTE only, not a default — see below

The real constraint: **3 of the 21 families have exactly one occurrence in the whole dataset** (Egregor, Ryuk, Vice Society), and 7 more have fewer than 10 (BlackMatter=2, Maze=5, DarkSide=5, Babuk=4, Black Basta=6, BlackCat=6, Cactus=6). No oversampling method — classical or generative — can help a class with one real example; there's nothing to interpolate or model a distribution from, and no CV split can ever place it in both train and test. Those 3 are excluded from the quantitative family task entirely (the framework does this automatically and prints the exclusion) and belong in the paper as qualitative case studies instead. For the rest, CTGAN specifically is a poor first choice: a conditional GAN needs hundreds of real rows per class to avoid mode collapse, and 5–6 real examples is much more likely to produce near-duplicate synthetic rows that look like they helped on a naive check without generalizing. Worth running as an explicit "does deep generative augmentation beat classical interpolation here" ablation once installed — not as the default path.

Three validation protocols, run in parallel

- **Stratified random k-fold** — the standard number, and the one most likely to be optimistic.
- **GroupKFold by `Channel_ID`** — no channel appears in both train and test. With up to 20 videos per channel and only 92 channels, a model can otherwise learn "this is CrowdStrike's narration style" instead of "this transcript discusses lateral movement." The gap between this score and the random-fold score *is* a result worth reporting on its own.
- **Temporal split** — train on pre-2024 (≈217 videos), test on 2024–2026. This is the honest version of "does this generalize," given how young most of the corpus is.

Phase 2

Uncertainty quantification

The point of this phase is comparing *calibration against the Phase 0 gold set*, not against the weak labels — a model that's perfectly calibrated against its own training signal has just learned to be confidently correlated with a regex, which isn't the claim you want in a paper.

EXPERIMENTS

Calibration	Reliability diagrams + Expected Calibration Error, per task, per model, scored against gold labels
Random Forest	tree-vote entropy / class-probability spread as a free uncertainty proxy
Transformer	MC-Dropout at inference and/or a 3–5 seed deep ensemble for predictive entropy
Headline metric	"confidently wrong" rate — fraction of gold-set errors made with >90% predicted confidence, compared across models

Phase 3

Conformal prediction

Split conformal prediction turns any of the Phase 1 classifiers into a set-valued predictor with a distribution-free coverage guarantee — a natural fit here precisely *because* the point labels are weak: instead of trusting a single predicted family, you get "the true family is in this set with 90% probability," which is a more honest thing to claim about noisy labels.

EXPERIMENTS

Single-label tasks	Split conformal on Dominant_Tactic and Platform_Signal at 90% target coverage — report empirical coverage and mean set size
Multi-label tasks	Per-label conformal calibration (or conformal risk control on Hamming/false-negative rate) for Family and Tool tasks
Conditional coverage stress-test	Recompute coverage restricted to: the gold subset only, post-2025 videos only, conference-channel videos only. If coverage collapses on any slice, that slice *is* the interesting result — it's telling you where the exchangeability assumption breaks, which links straight into Phase 4.

Phase 4

Out-of-distribution detection

Four OOD setups, each grounded in a real split already present in the data rather than a synthetic one.

SETUP	ID	OOD
Temporal emergence	train pre-2024	test videos mentioning families that only appear post-2023 (Qilin, DragonForce, Black Basta)
Channel-type shift	Vendor + DFIR channels	Conference + Independent-YouTuber channels
Leave-rare-family-out	families with n≥10	the 11 families with n<10 held out entirely from training
External non-security corpus	RansKnow transcripts	a general tech-YouTube corpus, as a sanity check that OOD scores actually spike on genuinely unrelated content

METHODS COMPARED, PER SETUP

Baseline	max-softmax-probability
Distance-based	Mahalanobis distance on pooled embeddings
Energy-based	energy score

Conformal-native	nonconformity score from Phase 3 as an OOD flag directly — ties the two phases together instead of treating them as separate chapters
Metric	AUROC and FPR@95%TPR for ID-vs-OOD separation

Phase 5

Confounds, counterfactuals & perturbation robustness

Three related diagnostics, all asking a version of the same question — *is the model reading the content, or reading something correlated with it* — and all requiring a trained Phase 1 model to interrogate, which is why this phase comes after the baseline ladder rather than alongside Phase 0's dataset-level audit.

5.1 – Confound checks

- **Length confound** — re-run family/tactic detection as mentions-per-1,000-words rather than raw counts. Longer videos have more chances to trip a keyword regardless of actual content; if the "detection rate" story changes under length normalization, raw counts were partly measuring video length.
- **Channel-identity leakage** — strip or mask channel-identifying phrases (channel name, host names, recurring intros) from transcripts and re-score the Phase 1 models. A large accuracy drop means the model was partly reading "which channel is this" rather than "what does this transcript describe."
- **Provider-stratified error analysis** — report every headline metric split by `Transcript_Provider` (ASR vs. official caption), using the Phase 0.4 jargon spot-check to interpret any gap.

5.2 – Counterfactual explanations

For a trained text classifier, a minimal-word-removal counterfactual — "what's the smallest edit that flips this prediction from Family=Play to Family=None" — is a direct, model-level test of whether a downstream classifier learned real ransomware vocabulary or inherited the same kind of spurious-word correlation the regex baseline

had (Phase 0.2's Play/Hive/Cactus collisions). If a model's counterfactual for a `Play` prediction bottoms out on removing the word "play" itself with no other family-specific vocabulary in the minimal edit set, that's independent evidence the noise propagated into the model rather than being trained away.

> **EXPERIMENT 5.2**
>
> | | |
> |---|---|
> | Method | Greedy/beam word-removal search — mask candidate tokens, keep removals that move predicted probability toward the counterfactual target class, repeat until the prediction flips or a removal budget is hit |
> | Why not MiCE/Polyjuice | generative counterfactual methods are built for producing fluent edited text; here the edit itself (which words, how many) is the object of interest, not fluency — a dependency-light search is the right tool, not a generative model |
> | Priority targets | every video where the rule-based baseline's `Play` / `Hive` / `Cactus` tag was flagged as a likely false positive in Phase 0.2 — do the trained models make the same mistake? |

5.3 – Perturbation-robustness sweep, and what "adversarial" means for this dataset

Counterfactual search above is *targeted* — hunting for the minimal edit that flips one specific prediction. This is the untargeted counterpart: perturb inputs within a semantic-preserving budget and measure how much predictions move, without aiming at a specific flip.

- **Simulated ASR noise** — inject Whisper-realistic errors (phonetically-similar word substitution, low-confidence-token deletion) into the 74 official-caption transcripts and measure prediction drift. This directly operationalizes Finding B (92.7% of the corpus is ASR-transcribed) as a controlled experiment instead of an aggregate-rate observation.
- **Synonym substitution** — replace non-jargon words with synonyms; a robust model's predictions shouldn't move much, since surface wording changed but technical content didn't.

On the word "adversarial" specifically: there's no attacker submitting transcripts to evade this classifier in production, so a full gradient-based adversarial-attack

framework (TextFooler-style, requiring a differentiable model) is disproportionate to what this dataset paper needs and is not recommended here. The perturbation sweep above is the dataset-appropriate proxy — it targets the actual sources of input noise this corpus has (ASR errors, channel-style variation), not a hypothetical adaptive adversary. Combined with Phase 4's temporal-emergence and leave-rare-family-out OOD setups, which already cover "how does this handle families that didn't exist in training," these three pieces are the complete answer to "how do you handle evolving, new, and adversarial patterns" — not three separate open questions.

REFERENCE

Metrics by task

PHASE	METRIC
Baselines (multi-label: Family, Tool)	macro-F1, per-class precision/recall (report the long tail, not just macro average)
Baselines (multi-class: Tactic, Platform)	macro-F1, confusion matrix
Baselines (binary screen)	PR-AUC (base rate is unbalanced — 52% zero-family)
Uncertainty	ECE, reliability-diagram slope, confidently-wrong rate
Conformal	empirical coverage vs. target, mean/median set size, conditional coverage by slice
OOD	AUROC, FPR@95%TPR, per setup and per method

SECTION 12

Suggested paper structure

1. **Introduction** — the gap: no open, transcript-level ransomware knowledge corpus with MITRE-aligned annotations; RansKnow's construction pipeline as the

contribution.

2. **Related work** — ransomware threat-intel datasets, weak/distant supervision in security NLP, conformal prediction and OOD detection in NLP.
3. **Dataset construction** — the fetch pipeline, keyword-matched channel curation, the Whisper/caption transcript cascade (own Finding B as a methods-section admission, not a limitations-section footnote).
4. **Label-quality audit** — Findings A–C, the alias re-audit, the gold-subset construction and inter-annotator agreement. This section is the paper's methodological backbone.
5. **Task definitions & baselines** — Phase 1, all three CV protocols reported side by side.
6. **Uncertainty & conformal prediction** — Phases 2–3.
7. **Out-of-distribution robustness** — Phase 4.
8. **Interpretability & perturbation robustness** — Phase 5: confound checks, counterfactual explanations, and the ASR/synonym perturbation sweep as the dataset-appropriate answer to evolving and adversarial-style robustness.
9. **Limitations & ethics** — YouTube-sampling bias, ASR noise, weak-label ceiling, dual-use considerations for a ransomware-tactics corpus.
10. **Conclusion** — release the gold-eval subset and audit code as the dataset's second artifact, not just the transcripts.

SECTION 13

Sequencing

PHASE	DEPENDS ON	BLOCKS
0 — data & label audit	nothing	everything below
1 — baseline ladder	Phase 0 gold set	2, 3, 4
2 — uncertainty	Phase 1 trained models	3 (shares calibration infra)

PHASE	DEPENDS ON	BLOCKS
3 — conformal prediction	Phase 1 models, Phase 2 calibration	4 (nonconformity score reused)
4 — OOD detection	Phase 1 embeddings, Phase 3 scores	writing
5 — confound checks	Phase 1 models	can run in parallel with 2–4

Phase 0 is the only true bottleneck — nothing downstream is trustworthy without the gold subset, so it's worth front-loading even though it's the least glamorous phase to write about. Phase 5 can run anytime after Phase 1 finishes and doesn't block anything.

RansKnow experimental plan · grounded in `outputs/Knowledge_Agent_Features_1034.csv` as of this dataset version (92 channels, 1,128 videos, 1,034 transcripts).